

Analyzing Enemy Behavior Difficulty: An Automated Framework Exploiting Behavior Trees

David Ponton

dept. of computer science and mathematics
Université du Québec à Chicoutimi
 Chicoutimi, Canada
 dponton@etu.uqac.ca

Hugo Tremblay 

dept. of computer science and mathematics
Laboratory of Research in Modelling, Graphs
and Optimization
Université du Québec à Chicoutimi
 Chicoutimi, Canada
 hugo_tremblay2@uqac.ca

Yannick Francillette 

dept. of computer science and mathematics
Université du Québec à Chicoutimi
 Chicoutimi, Canada
 yannick_francillette@uqac.ca

Bruno Bouchard 

dept. of computer science and mathematics
Université du Québec à Chicoutimi
 Chicoutimi, Canada
 bruno.bouchard@uqac.ca

Abstract—Balancing difficulty is a cornerstone of effective video game design, directly influencing player engagement and satisfaction. Among the various type of difficulty, comprehensive difficulty of in-game enemies poses a unique challenge due to the intricate rules and parameters defining their behavior. This paper addresses the gap in tools for automatically assessing enemy difficulty, particularly during early development stages where machine learning (ML)-based approaches are often impractical. We propose a novel framework for the automatic measurement of the comprehensive difficulty of enemies modeled using Finite-State Machines (FSM) or Behavior Trees (BT). By leveraging the standardized structure of BTs, our approach calculates difficulty based on two key factors: the amount of information, defined by the number of states, and the structural complexity, quantified using cyclomatic complexity. Additionally, we introduce a method for converting FSM models into BTs, ensuring compatibility with our assessment framework. To validate our method, we conducted experiments using enemy models from Super Mario, demonstrating that our automated algorithm produces results consistent with the intended difficulty levels designed by game developers. By reducing reliance on extensive playtesting, this tool empowers developers to iteratively and objectively evaluate enemy difficulty, even in the early stages of development. Our findings underscore the potential of graph-based methods to revolutionize enemy design workflows, paving the way for more efficient and precise balancing in video game development. This framework also highlights the feasibility of integrating such tools into commercial game engines like Unreal Engine 5 to streamline development processes.

Index Terms—Video game, Enemy Design, Automatic difficulty assessment, Comprehensive difficulty, Behavior Trees, Finite-State Machines

I. INTRODUCTION

In video games, balancing the level of difficulty is a critical factor in enhancing the player's overall experience [1]. According to Frampton [2], difficulty can be classified into three main categories: motor or executive difficulty, strategic difficulty, and comprehensive difficulty. Motor or executive difficulty challenges the player's physical abilities, such as dexterity, precision, and reaction speed when using game controllers

[3]. Strategic difficulty, on the other hand, tests the player's mental and logical capabilities, requiring them to make effective strategic and tactical decisions during gameplay. Finally, comprehensive difficulty refers to the challenges associated with fully understanding the game's core rules, objectives, and parameters. For game designers, comprehensive difficulty is particularly challenging to calibrate due to the vast array of rules and parameters that define a game. This complexity makes it one of the most significant hurdles in the game design process [4]. Among the core elements contributing to a game's difficulty, one of the most visible and impactful is the design of the player's enemies.

The behavior of in-game enemies plays a pivotal role in game development, profoundly influencing player engagement and satisfaction [5]. Regardless of the genre be it a first-person shooter, role-playing game, or platformer designing intelligent and challenging enemy AI is essential for enhancing the overall gameplay experience. Many factors contribute to the challenge of understanding an enemy's behavior [6], including the variety of patterns it can exhibit, the range of weapons it uses, its ability to regenerate, and more. During the development process, fine-tuning the difficulty level of enemies is crucial for creating an engaging player experience [7]. Enemy difficulty serves as a core mechanism for delivering challenges, demanding careful calibration by designers to align with the game's intended objectives. A well-designed game achieves this balance by introducing enemies in a thoughtfully paced manner and fostering a sense of meaningful progression throughout the gameplay.

Crafting an enemy is undeniably a complex and labor-intensive process. Toward the end of development, designers typically rely on user playtesting to fine-tune the game's difficulty and achieve optimal balance. Some researchers have proposed the integration of machine learning (ML) methodologies to assist game developers in this process [8]. However, the effectiveness of ML approaches depends heavily on the

availability of extensive datasets generated through playtesting data that is often limited or entirely absent during the early stages of development. ML solutions excel in refining games at later stages or in games that undergo continuous updates, such as *World of Warcraft*¹, and in generating procedural content based on established patterns, such as new enemies. Yet, in the early phases of game development when designers are actively adjusting mechanics, experimenting with features, and tweaking parameters that influence player experience and challenge ML methods are often unusable. This highlights the need for tools based on non-ML approaches to provide objective, automatic measurements of enemy difficulty and to assess how new enemy designs impact the comprehensive difficulty of the game [9]. That kind of tool, integrated as a plugin inside a commercial game engine such as Unreal Engine², could really help optimizing the development process.

In the scientific literature, relatively few research teams have focused specifically on the problem of automatically assessing the difficulty of enemies in games [4], [6], [10]. Furthermore, there has been limited progress in proposing systematic, formal definitions and algorithmic methods for objectively evaluating enemy difficulty. The behavior of enemies, in the vast majority of computer games, is typically modeled using Finite-State Machines [11] or Behavior Trees [12]. For example, in Unreal Engine 5, developers can easily use a graphical tool to design an enemy's behavior with a simple Behavior Tree structure. This widespread standardization in enemy behavior modeling presents an opportunity to define and develop tools for automatically assessing the comprehensive difficulty of an enemy [4].

In this paper, we introduce a novel method for the automatic measurement of the comprehensive difficulty of enemies modeled with Finite-State Machines or Behavior Trees. Our approach leverages the standardized structure of Behavior Trees to calculate enemy difficulty by utilizing graph properties [13]. Additionally, we propose a method to convert enemy FSM models into Behavior Trees, enabling the computation of relative comprehensive difficulty based on two key factors: the amount of information, quantified by the number of states, and the complexity of the transition structure, measured using cyclomatic complexity. This framework allows for an objective comparison of enemy difficulty levels, significantly reducing the reliance on playtesters. We detail the proposed model, its implementation, and the initial results of an experiment conducted using state-machine models of enemies from *Super Mario*. The promising findings demonstrate that our automated assessment algorithm produces results that are highly consistent with the intended design of the game.

II. RELATED WORK

The scientific literature contains numerous studies addressing the automatic measurement of difficulty in games [8], [9], [14]–[19]. However, the majority of these works focus

primarily on assessing executive or motor challenges. As noted in the introduction, these challenges are defined by the degree of spatial and temporal precision required for players to execute actions successfully. Our previous work specifically tackled this issue [9]. In that study, we proposed an innovative method for the formal calculation of static danger zones within game levels, coupled with the analysis of enemy movement patterns using simulated pheromones.

Regarding the challenge of measuring comprehensive difficulty, which encompasses understanding a game's core rules, objectives, and parameters [2], very few studies have addressed this crucial issue. Specifically, the literature on assessing the comprehensive difficulty of in-game enemies, a key component of overall comprehensive difficulty, remains limited [4]. Pichit et al. [6] proposed a framework for evaluating battle performance, which can be interpreted as a form of difficulty assessment for AI implementations. Their approach combines Finite-State Machines, search algorithms, and rule-based heuristics [20] to design AI agents that mimic human players in terms of battle performance. They utilized playtesting data to develop an AI capable of achieving performance levels comparable to those of a standard player. This AI was then used to evaluate new enemies by analyzing its performance against them. For example, they measured metrics such as the number of moves made by the AI and the amount of damage taken. While this approach is innovative and automates the measurement of enemy difficulty, it relies on player data to create the AI. Consequently, it is not applicable during the early stages of game development. Additionally, this method does not provide a standardized framework for comparing the comprehensive difficulty of enemies across different games.

Zhang [10] recently conducted research addressing how to control perceived difficulty by adding enemy objects that do not actually increase the objective difficulty level. To explore this, they designed a simple shooting game featuring two types of "fake enemy bullets": Unreachable Bullets and Non-Collisional Bullets, neither of which could harm the player character. The experiment demonstrated that Non-Collisional Bullets effectively increased the perceived difficulty, whereas Unreachable Bullets did not. This study highlights novel techniques for manipulating perceived difficulty, providing a basis for comparative measurement of player perception. However, it does not offer a framework for analyzing the objective, actual difficulty of an enemy.

Viana et al. [21] addresses the challenge of designing enemies in Action-Adventure games, whose primary role is to hinder player progression by reducing life points. It presents an extended evolutionary approach for procedurally generating enemies with targeted difficulty levels. The method incorporates a MAP-Elites algorithm [22] to produce a diverse range of high-quality enemies. Computational experiments demonstrated that the algorithm efficiently converges, generating most enemies within the MAP-Elites framework in under a second. Additionally, player testing with a game prototype featuring the generated enemies revealed that players enjoyed the levels and successfully perceived enemies as easy, medium,

¹<https://worldofwarcraft.blizzard.com/>

²<https://www.unrealengine.com/en-US/>

or hard, validating the approach's effectiveness. Nevertheless, their proposal still does not address the issue of formalizing the measurement of enemy difficulty.

Finally, we presented a preliminary approach for analyzing enemy difficulty [4]. In that work, we proposed a method for automatically measuring the comprehensive difficulty of game enemies using only state machines. Alongside the model, we shared the results of an initial exploratory experiment, demonstrating the potential of our approach and its feasibility for integration as a plugin in game engines like Unreal Engine. This paper builds directly on last year's work by introducing a novel framework based mainly on Behavior Trees, presenting new experimental results, and proposing a method for converting enemy state machines into BTs. These advancements are significant, given that most modern games use BTs to model enemy behavior.

III. MATHEMATICAL FRAMEWORK

The proposed framework aims to take advantage of the standardized models of enemies behavior in many games with Finite-State Machines and behavior trees to expand on our previous work for evaluating comprehensive difficulty based on properties of the underlying graph structure. We first define a canonical representation of an enemy behavior by converting the associated FSM to a BT. We then apply metrics to the resulting structure in order to obtain a difficulty score for that particular enemy. Note that our method assumes the FSMs are known, either through design documentation or manual reconstruction based on in-game observations.

A. Converting FSM to BT

For the sake of completeness, we recall some basic definitions and properties of FSM and BT, as presented by various authors in [23]–[25].

Formally, a FSM is a 5-tuple

$$M = (Q, \Sigma, \delta, q_0, F)$$

where Q is a finite set of states (e.g. hiding, moveTo), Σ is a finite alphabet of events (e.g. spawn, fall), $\sigma : Q \times \Sigma \rightarrow Q$ is a transition function, $q_0 \in Q$ is the initial state (i.e. the creation of the object) and $F \subseteq Q$ is the set of accepting states (i.e. the destruction of the object). Remark that FSM specifically representing enemy behavior are constructed such that $\#F = 1$, $q_0 \notin F$ and $\epsilon \in \Sigma$. We refer the G_M , the underlying labeled graph of M , as M itself when the context is clear.

Behavior trees are directed rooted trees where the leaves are called “execution nodes”, and the internal nodes are “control flow nodes”. Execution nodes can either be Actions or Conditions, while control flow nodes can be one of four types: Sequence, Fallback, Parallel, or Decorator. BTs operate using signals called ticks, which traverse the tree from the root, directed by control flow nodes. Ticks return one of three values: Success, Failure, or Running. Action nodes perform tasks and return Running, Success, or Failure based on task completion. Condition nodes check logical propositions and

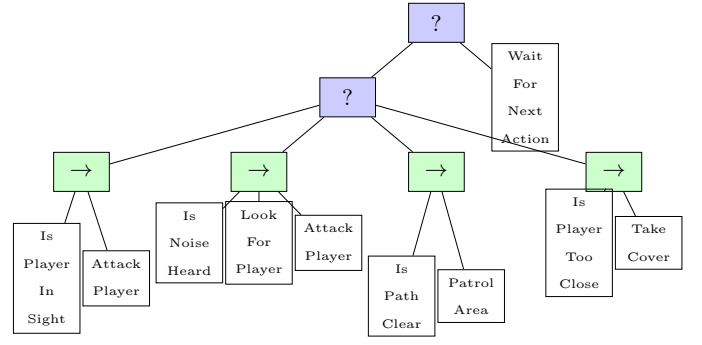


Fig. 1: Behavior tree of a typical soldier enemy in Call of Duty.

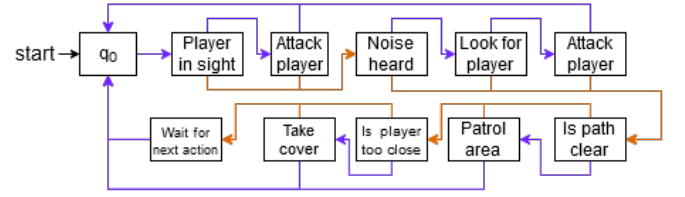


Fig. 2: FSM obtained by applying Algorithm 1 to the BT of Figure 1.

return Success or Failure accordingly. Sequence nodes (AND operator) pass ticks to children in sequence, stopping on Running or Failure. Fallback nodes (OR operator) pass ticks until a child returns Success or Running. Parallel nodes distribute ticks to children, returning Success or Failure based on a set threshold. Decorator nodes modify child return values or control tick propagation. Finally, nodes can also be configured to remember outcomes, marked with an asterisk. Figure 1 shows a typical BT for a soldier enemy in *Call of Duty*.

From a theoretical point of view, FSM and BT are equivalent (i.e. have the same descriptive power). However, BTs modularity is better suited for describing behavior in video games. Given a BT representing the behavior of an enemy, we obtain its canonical representation by first converting it into a FSM, before applying a new conversion to obtain the canonical BT. Remark that our difficulty assessment metric is applicable to any behavior tree regardless. We merely use this canonical form to derive and study mathematical properties of BTs. Both conversion processes are described in the following Algorithms 1 and 2.

As an example, consider the BT of the enemy soldier of Call of Duty represented in Figure 1. First applying Algorithm 1 gives the FSM of Figure 2. Then, this FSM is transformed into a BT using Algorithm 2. The resulting tree is illustrated in Figure 3.

B. Difficulty assessment metric

For sake of completeness, we briefly recall the metric for measuring comprehensive difficulty, as presented in [4]. Given a graph structure G , we define its comprehensive difficulty as the amount of information contained in G along with the

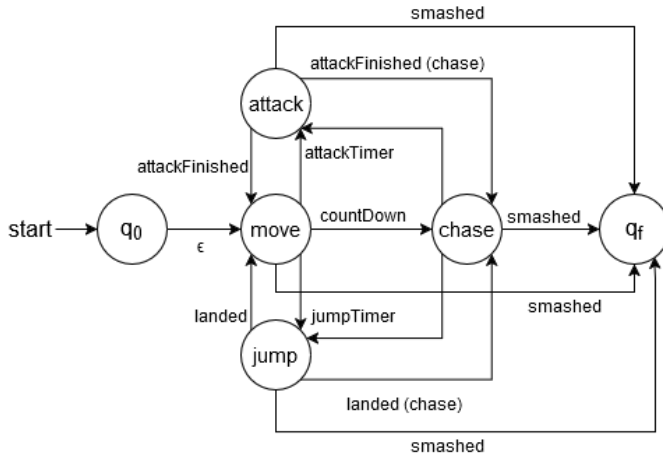


Fig. 4: FSM for the *Hammer bro.* enemy of Super Mario bros.

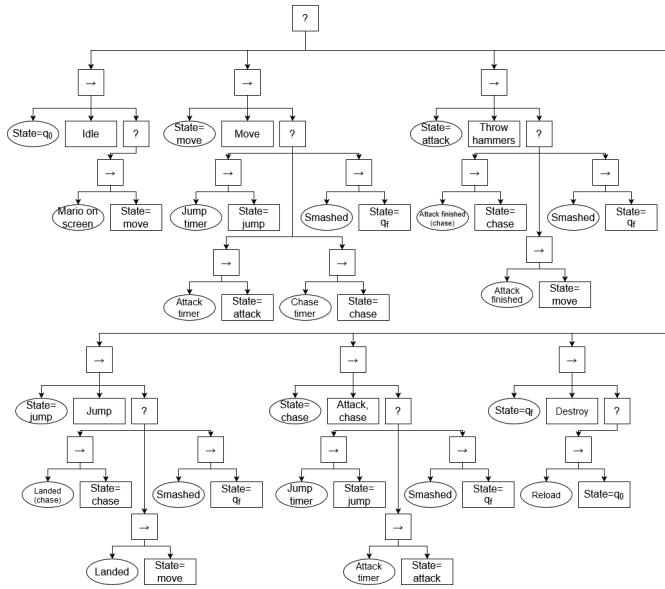


Fig. 5: Canonical BT for the *Hammer bro.* enemy of Super Mario bros.

structure and ordered them according to difficulty. The results are in Table I.

This follows our intuition that *Goombas*, being only able to walk laterally, are the simplest enemy. On the other end of the spectrum, *hammer bro*, with his complex movement and attack patterns, should be considered one of the more difficult enemies to comprehend. Ideally, enemies should appear in game more or less by their order of difficulty. Of course, one must take into account other factors such as other difficulty types and narrative choices. Yet, based on comprehensive difficulty alone, we arrive at a ranking of the enemies that matches their introduction order in-game. Figure 6 represents the comprehensive difficulty of each enemy relative to their first appearance in the game. Note that our metric remains relevant even for games in which the order of enemy appearance does not reflect their actual difficulty (e.g., *Elden Ring*), as

TABLE I: Comprehensive difficulty for the FSM and BT of several enemies from Super Mario bros.

Enemy	FSM	BT
Gomba	9	140
Koopa	20	328
Blooper	20	360
Lakitu	20	360
Parakoopa	30	510
Hammer Bro.	60	1050

it still provides valuable insights for designers—for instance, when determining optimal enemy placement within individual levels.

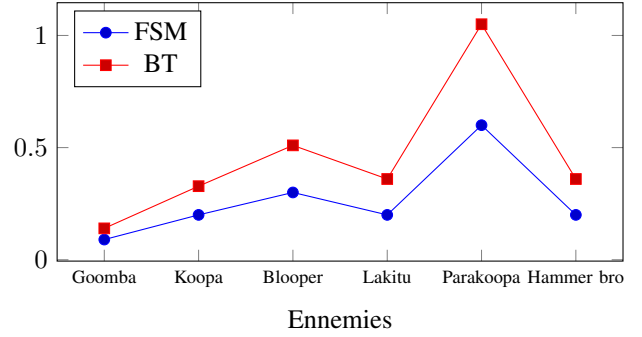


Fig. 6: Comprehensive difficulty curve of the enemies in order of first appearance in Super Mario bros.

In order to compare the difficulty score of both FSMs and BTs, we adjusted the difficulty score to the interval $[0, 1]$, by dividing by 10 and 100 respectively. The adjusted comprehensive difficulty scores are in Table II.

TABLE II: Adjusted comprehensive difficulty for the FSM and BT of several enemies from Super Mario Bros.

Enemy	FSM	BT
Gomba	0.09	0.14
Koopa	0.2	0.328
Blooper	0.2	0.36
Lakitu	0.2	0.36
Parakoopa	0.3	0.51
Hammer Bro	0.6	1.05

Figure 7 offers a visual representation of the relationship of both adjusted difficulty curves. As the figure suggest, there is a near-perfect correlation between the two datasets, calculated at 99.9%. This result is not surprising since each BT is equivalent to its corresponding FSM. It nevertheless reveals an interesting fact: both models of representation could be used in parallel during development. Even not accounting for the adjusted score, the "raw" comprehensive difficulty score between equivalent FSM and BT differs by a factor of 10.

V. CONCLUSION AND FUTURE WORK

This paper introduces a novel framework for the automatic assessment of the comprehensive difficulty of in-game enemies modeled using Finite-State Machines or Behavior Trees. By leveraging graph properties such as the number of states and

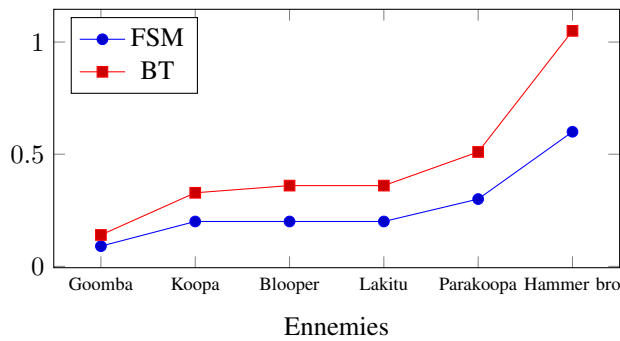


Fig. 7: Comprehensive difficulty curve of the enemies in ascending difficulty order in Super Mario bros.

cyclomatic complexity, the proposed method provides an objective and systematic approach to evaluating enemy difficulty. Our experimental validation with enemy models from Super Mario demonstrates that this method aligns closely with the intended design difficulty, significantly reducing the reliance on playtesting. Furthermore, the framework is compatible with commercial game engines like Unreal Engine 5, enabling practical integration into existing workflows and facilitating iterative development processes.

Despite these promising results, several limitations remain. First, the method primarily focuses on structural aspects of enemy behavior, which may not fully capture dynamic interactions with players, such as adaptability or learning mechanisms in enemy AI. Second, the framework assumes standardized behavior modeling (FSMs or BTs), potentially limiting its applicability to games with custom or hybrid AI architectures. Additionally, while the approach is effective for comprehensive difficulty, it does not address other dimensions of difficulty, such as motor or strategic challenges.

Future work will address these limitations by extending the framework to account for player-enemy interactions and incorporating additional difficulty dimensions. Moreover, we aim to explore how this method could be adapted for more complex AI systems, such as those utilizing machine learning or procedural generation. Other models for behavior trees would help in modeling and quantifying the inherent difficulty of understanding randomness in enemy behavior. Finally, conducting large-scale validation with diverse datasets across various genres will be crucial to generalizing the approach and further enhancing its utility in real-world game development.

REFERENCES

- [1] K. Eberhart, "Difficulty levels and why you should never use them," *GameDeveloper.com*, pp. 1–5, 2016.
- [2] R. Frampton, "A new taxonomy of difficulty: A call to reconsider how we talk about difficulty in games," *GameDeveloper.com*, pp. 1–7, 2021.
- [3] G. Azevedo, M. L. Gunsch, M. G. Lacerda, H. D. O. Peixoto, L. C. L. F. Borges, and E. P. D. S. Nunes, "Requirements gathering regarding fine motor skills, adaptive difficulty and executive functions for a game in development for therapy sessions with autistic children to encourage collaboration," in *Proceedings of the 2023 ACM International Conference on Interactive Media Experiences Workshops*, ser. IMXw '23. New York, NY, USA: Association for Computing Machinery, 2023, p. 32–37. [Online]. Available: <https://doi.org/10.1145/3604321.3604345>
- [4] Y. Francillette, H. Tremblay, and B. Bouchard, "Automated difficulty assessment model for comprehensive difficulty in games," in *2024 IEEE Gaming, Entertainment, and Media Conference (GEM)*, 2024, pp. 1–6.
- [5] T. Steffen, "Mastering enemy behavior in game development," *Medium : Human stories ideas (Blog)*, pp. 1–2, 2023.
- [6] P. Promsutipong and V. Kotrajaras, "Enemy evaluation ai for 2d action-platform game," in *2017 14th International Joint Conference on Computer Science and Software Engineering (IJCSE)*, 2017, pp. 1–6.
- [7] J. T. Kristensen, A. Valdivia, and P. Burelli, "Statistical modelling of level difficulty in puzzle games," in *2021 IEEE Conference on Games (CoG)*. IEEE Press, 2021, p. 1–8.
- [8] N. Fisher and A. K. Kulshreshth, "Exploring dynamic difficulty adjustment methods for video games," *Virtual Worlds*, vol. 3, no. 2, pp. 230–255, 2024.
- [9] Y. F. H. T. B. S. L. M. R. J. Linard, "Automated difficulty assessment model for platformer games: A comprehensive approach," in *2023 IEEE CTSoc Gaming, Entertainment and Media conf.*, 2023, pp. 1–6.
- [10] J. Zhang, "Directly controlling the perceived difficulty of a shooting game by the addition of fake enemy bullets," in *Extended Abstracts of the 2021 CHI Conference on Human Factors in Computing Systems*, ser. CHI EA '21. New York, NY, USA: Association for Computing Machinery, 2021.
- [11] D. Jagdale, "Finite state machine in game development," *International Journal of Advanced Research in Science, Communication and Technology (IJARSCT)*, pp. 1–7, 2021.
- [12] Y. A. Sekhavat, "Behavior trees for computer games," *International Journal on Artificial Intelligence Tools*, vol. 26, no. 02, p. 1730001, 2017. [Online]. Available: <https://doi.org/10.1142/S0218213017300010>
- [13] D. Jallof, *Graph Algorithms for AI in Games*. Packt, 2017.
- [14] M.-V. Aponte, "Difficulty in videogames: An experimental validation of a formal definition," in *Proceedings of the 8th International Conference on Advances in Computer Entertainment Technology*, ser. ACE '11. New York, NY, USA: Association for Computing Machinery, 2011.
- [15] P. D. Paraschos and D. E. Koulouriotis, "Game difficulty adaptation and experience personalization: A literature review," *International Journal of Human-Computer Interaction*, vol. 39, no. 1, pp. 1–22, 2023.
- [16] M. Ouellette, L. Breeding, and C. Clark, "Using applied cognitive load theory and difficulty analysis for educational game design for understanding and transference of literacy skills in adults," in *Proc. of the 14th Int. Conf. on the Foundations of Digital Games*, ser. FDG '19. New York, NY, USA: Association for Computing Machinery, 2019.
- [17] Y. J. Kim and J. A. Ruipérez-Valiente, "Data-driven game design: The case of difficulty in educational games." Berlin, Heidelberg: Springer-Verlag, 2020, p. 449–454.
- [18] P. Kaimara, "Development of a scale for measuring the learning experience in serious games," *Digital Culture Audiovisual Challenges, Interdisciplinary Creativity In Arts and Technology*, pp. 1–7, 2019.
- [19] J. L. Aurentz, A. M. Navarro, and D. R. Insua, "Learning the rules of the game: An interpretable ai for learning how to play," *IEEE Transactions on Games*, vol. 14, no. 2, pp. 253–261, 2022.
- [20] E. Suryawati and D. H. Widyantoro, "Combination of heuristic, rule-based and machine learning for bibliography extraction," in *2017 5th Int. Conf. on Instrumentation, Communications, Information Technology, and Biomedical Engineering (ICICI-BME)*, 2017, pp. 276–281.
- [21] B. M. F. Viana, L. T. Pereira, and C. F. M. Toledo, "Illuminating the space of enemies through map-elites," in *2022 IEEE Conference on Games (CoG)*. IEEE Press, 2022, p. 17–24. [Online]. Available: <https://doi.org/10.1109/CoG51982.2022.9893621>
- [22] T. Anne and J.-B. Mouret, "Parametric-task map-elites," in *Proceedings of the Genetic and Evolutionary Computation Conference*, ser. GECCO '24. New York, NY, USA: Association for Computing Machinery, 2024, p. 68–77. [Online]. Available: <https://doi.org/10.1145/3638529.3653993>
- [23] J. L. Gross, J. Yellen, and M. Anderson, *Graph theory and its applications*. Chapman and Hall/CRC, 2005.
- [24] M. Sipser, *Introduction to the Theory of Computation*, 3rd ed. Course Technology, 2013.
- [25] M. Colledanchise and P. Ögren, *Behavior trees in robotics and AI: An introduction*. CRC Press, 2018.
- [26] T. J. McCabe, "A complexity measure," *IEEE Trans. Softw. Eng.*, vol. 2, no. 4, p. 308–320, Jul. 1976.